情報理工学実験 II

# ネットワークプログラミング


学部学科　：　工学部 情報エレクトロニクス学科

学年　：　学士 3 年

学生番号　：　02246910

学部番号　：　ICT26903

学生氏名　：　吉野 拓人

メールアドレス　：　takuto.yoshino.l3@elms.hokudai.ac.jp


提出日：2026 年 11 月 12 日

# 目次


目次 . . . . . 1

課題 1-A . . . . . 2

課題 1-B . . . . . 4

課題 1-C . . . . . 7

課題 2-D . . . . . 9

課題 2-E . . . . . 12

課題 3-1 . . . . . 17

課題 3-2 . . . . . 23

課題 4-1 . . . . . 25

課題 4-2 . . . . . 28

発展課題 4-3 . . . . . 31

課題 4-4 . . . . . 33

課題 4-5 . . . . . 35

課題 4-6 . . . . . 37

課題 4-7 . . . . . 39

課題 4-8 . . . . . 44

# 課題 1-A

## 課題内容

IPv4・IPv6 の相手に ICMP echo request を送り，応答の有無を確認した.

## 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
(IPv4) tcpdump -i eth0 -s 0 -w ping4.pcap
(IPv6) tcpdump -i eth1 -s 0 -w ping6.pcap
```

2. キャプチャ中に，別のターミナルで ping, ping6 を実行

```
(IPv4) ping -c 5 172.27.0.1
(IPv6) ping6 -c 5 www.docomo.ne.jp
```

3. ping の終了後，キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

IPv4・IPv6 の tcpdump の実行結果と，ping・ping6 の実行結果をそれぞれ出力 1，出力 2，出力 3，出力 4 に示した.

出力 1: tcpdump(IPv4) の実行結果

```
guest@host82:~$ tcpdump -i eth0 -s 0 -w ping4.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C14 packets captured
14 packets received by filter
0 packets dropped by kernel
```

出力 2: tcpdump(IPv6) の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w ping6.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C132 packets captured
146 packets received by filter
0 packets dropped by kernel
```

出力 3: ping(IPv4) の実行結果

```
guest@host82:~$ ping -c 5 172.27.0.1
PING 172.27.0.1 (172.27.0.1) 56(84) バイトのデータ
64 バイト応答 送信元 172.27.0.1: icmp_seq=1 ttl=64 時間=0.029 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=2 ttl=64 時間=0.024 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=3 ttl=64 時間=0.050 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=4 ttl=64 時間=0.020 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=5 ttl=64 時間=0.026 ミリ秒

--- 172.27.0.1 ping 統計 ---
送信パケット数 5，受信パケット数 5，パケット損失 0%，時間 4097 ミリ秒
rtt 最小/平均/最大/mdev = 0.020/0.029/0.050/0.010 ミリ秒
```

出力 4: ping6(IPv6) の実行結果

```
guest@host82:~$ ping6 -c 5 www.docomo.ne.jp
PING www.docomo.ne.jp(2600:9000:2772:3600:7:a2b3:f240:93a1 (2600:9000:2772:3600:7:a2b3:f240:93a1)) 56 デー
↪  タ長 (byte)
^C
--- www.docomo.ne.jp ping 統計 ---
送信パケット数 5，受信パケット数 0，パケット損失 100%，時間 4100 ミリ秒
```

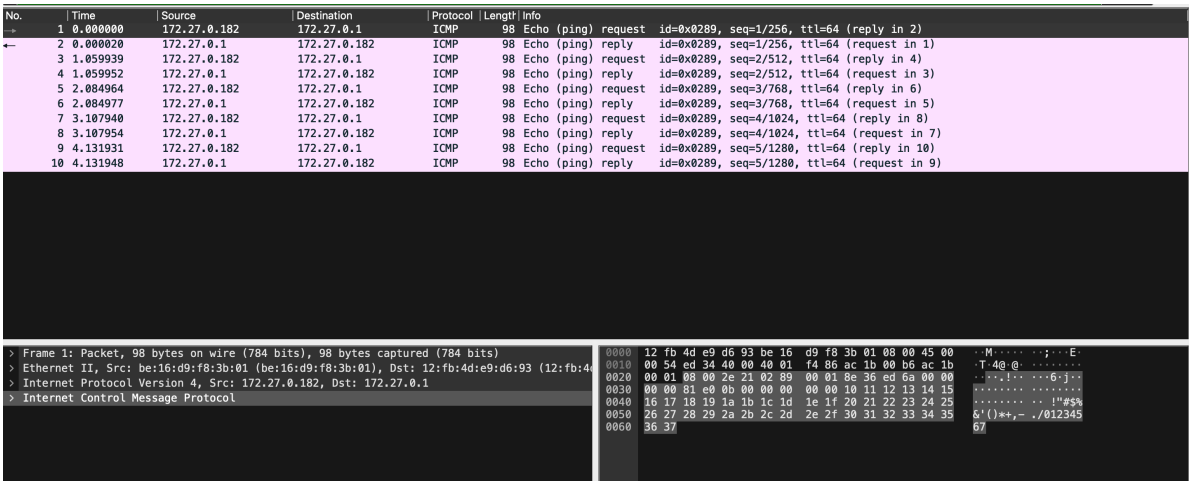



図 1: IPv4 の ping を Wireshark で読み取った結果

IPv4 では，送信元 172.27.0.182 から 172.27.0.1 に向けて ICMP echo request が 5 回送信され，それぞれに対する echo reply が確認できた．ping の結果も 5 パケット送信，5 パケット受信であり，パケット損失はなかった．平均往復時間は 0.029 ミリ秒であった．図 1 でも，request と reply が交互に並んでいることが確認できる．

IPv6 では www.docomo.ne.jp の名前解決はでき，2600:9000:2772:3600:7:a2b3:f240:93a1 宛ての echo request が 5 回送信されたが，echo reply は確認できなかった．ping の結果は 5 パケット送信，受信 0 であり，図 2 にも request のみが記録されていた．学外との ICMP 通信がネットワーク上で制限されている可能性があるため，この結果だけから通信相手の停止とは判断できない．

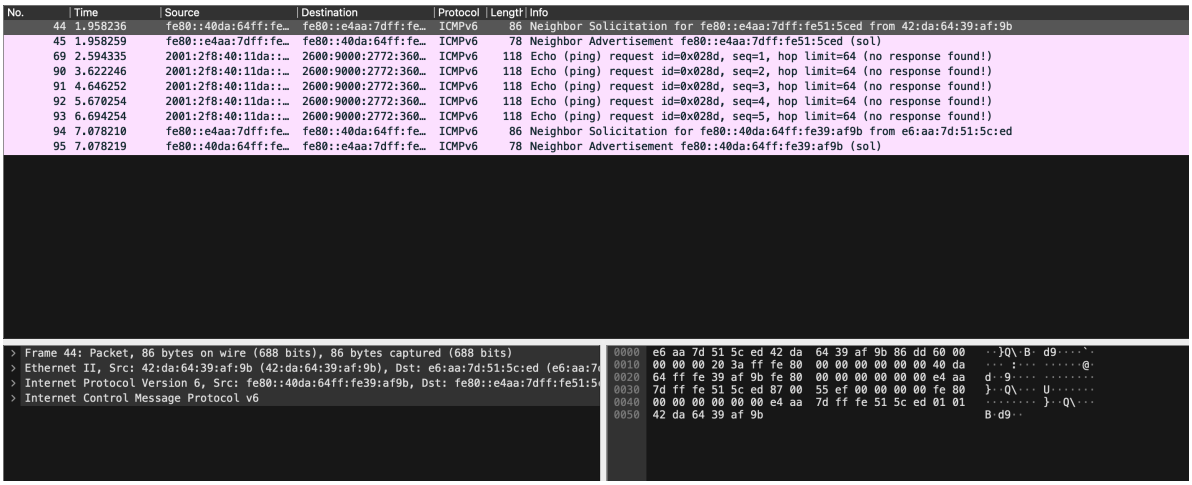



図 2: IPv6 の ping を Wireshark で読み取った結果

# 課題 1-B

## 課題内容

IPv4 では ARP，IPv6 では近隣探索で相手の MAC アドレスを調べ，そのパケットのやり取りを確認した．

## 実験方法

1. 実行前の近隣キャッシュを記録

```
ip ne
```

2. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w B.pcap 'not port 4200'
```

3. キャプチャ中に，別のターミナルで arping，ndisc6 を実行

```
(IPv4) arping -c 3 172.28.0.200
(IPv6) ndisc6 2001:2f8:40:11da::200 eth1
```

4. 実行後に近隣キャッシュを再度記録

```
ip ne
```

5. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

tcpdump の実行結果，実行前後の近隣キャッシュ，arping と ndisc6 の実行結果をそれぞれ出力 5，出力 6，出力 7，出力 8，出力 9 に示した．

出力 5: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w B.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C498 packets captured
500 packets received by filter
0 packets dropped by kernel
```

出力 6: 実行前の近隣キャッシュ

```
guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed REACHABLE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
```

出力 7: arping(IPv4) の実行結果

```
guest@host82:~$ arping -c 3 172.28.0.200
ARPING 172.28.0.200 送信元 172.28.0.182 eth1
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.742 ミリ秒
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.522 ミリ秒
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.520 ミリ秒
3 プローブを送信 (1 ブロードキャスト)
3 応答を受信
```

出力 8: ndisc6(IPv6) の実行結果

```
guest@host82:~$ ndisc6 2001:2f8:40:11da::200 eth1
Soliciting 2001:2f8:40:11da::200 (2001:2f8:40:11da::200) on eth1...
Target link-layer address: FE:89:01:C2:83:3A
from 2001:2f8:40:11da::200
```

出力 9: 実行後の近隣キャッシュ

```
guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::fc89:1ff:fec2:833a dev eth1 lladdr fe:89:01:c2:83:3a REACHABLE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router DELAY
```

IPv4 では，arping により 172.28.0.200 の MAC アドレスが FE:89:01:C2:83:3A であると分かった．出力 7 には「3 プローブを送信 (1 ブロードキャスト)」とあり，最初の 1 回がブロードキャストの ARP Request で，残りの 2 回は MAC アドレスが分かった後のユニキャストであると読み取れる．図 3 では，No.272 の ARP Request の宛先が Broadcast(ff:ff:ff:ff:ff:ff) で，“Who has 172.28.0.200? Tell 172.28.0.182” という内

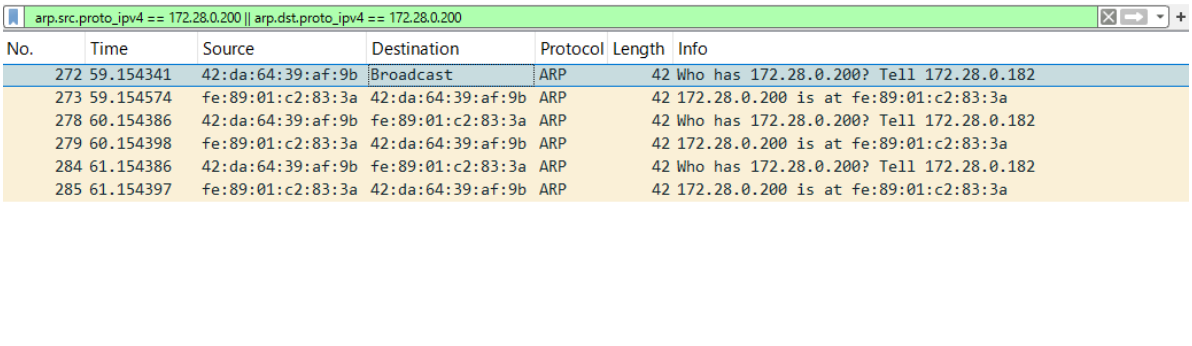



図 3: ARP のやり取りを Wireshark で読み取った結果

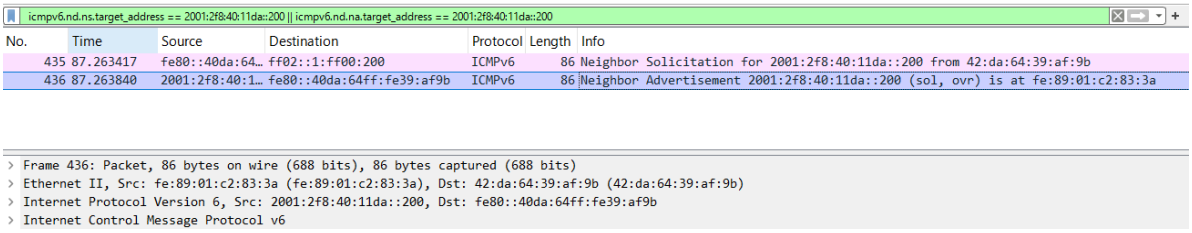



図 4: ND のやり取りを Wireshark で読み取った結果

容になっており，No.273 の ARP Reply では 172.28.0.200 が `fe:89:01:c2:83:3a` にいると返答している．続く No.278 と No.284 の Request は宛先が `fe:89:01:c2:83:3a` のユニキャストで，それぞれ No.279, No.285 で Reply が返っている．これは出力 7 の「1 ブロードキャスト」の内訳と一致する．

IPv6 では，ndisc6 により 2001:2f8:40:11da::200 の MAC アドレスが，IPv4 と同じ `FE:89:01:C2:83:3A` であると分かった．これは IPv4 と IPv6 の双方で，同一のホスト (sshd) の MAC アドレスが得られたことを示す．図 4 では，No.435 の Neighbor Solicitation(Type 135) が，宛先 `ff02::1:ff00:200`（2001:2f8:40:11da::200 に対する solicited-node マルチキャストアドレス）に送られている．その直後の No.436 の Neighbor Advertisement(Type 136) は，2001:2f8:40:11da::200 から要求元へユニキャストで返され，Target link-layer address として `fe:89:01:c2:83:3a` が通知されている．ARP がブロードキャストを使うのに対し，ND では対象のアドレスに対応するマルチキャストグループ宛てに Solicitation を送る点が異なる．

近隣キャッシュは，出力 6 と出力 9 を比べると，`fe80::fc89:1ff:fec2:833a` のエントリが新たに追加され，その MAC アドレスは上記と同じ `fe:89:01:c2:83:3a` であった．このアドレスは，MAC アドレスから EUI-64 形式で作られたリンクローカルアドレスである．一方，172.28.0.200 のエントリは実行前から存在し (STALE)，実行後も変化していなかった．それにもかかわらず，arping は No.272 でブロードキャストの Request を送信している．

# 課題 1-C

## 課題内容

LAN 内の TCP echo サーバに接続し，送信した文字列がそのまま返ってくることを，IPv4・IPv6 の双方で確認した．

## 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w C.pcap 'not port 4200'
```

2. キャプチャ中に，別のターミナルで telnet を実行し，適当な文字列を入力して返信を確認したのち，Ctrl+]，quit で終了する．IPv4 と IPv6 の双方で行う．

```
(IPv4) telnet 172.28.0.200 echo
(IPv6) telnet 2001:2f8:40:11da::200 echo
```

3. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

tcpdump の実行結果と，IPv4・IPv6 の telnet の実行結果をそれぞれ出力 10，出力 11，出力 12 に示した．

出力 10: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w C.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C500 packets captured
502 packets received by filter
0 packets dropped by kernel
```

出力 11: telnet(IPv4) の実行結果

```
guest@host82:~$ telnet 172.28.0.200 echo
Trying 172.28.0.200...
Connected to 172.28.0.200.
Escape character is '^]'.
hello
hello
world
world
^]
telnet> quit
Connection closed.
```

出力 12: telnet(IPv6) の実行結果

```
guest@host82:~$ telnet 2001:2f8:40:11da::200 echo
Trying 2001:2f8:40:11da::200...
Connected to 2001:2f8:40:11da::200.
Escape character is '^]'.
hoge
hoge
huga
huga
^]
telnet> quit
Connection closed.
```

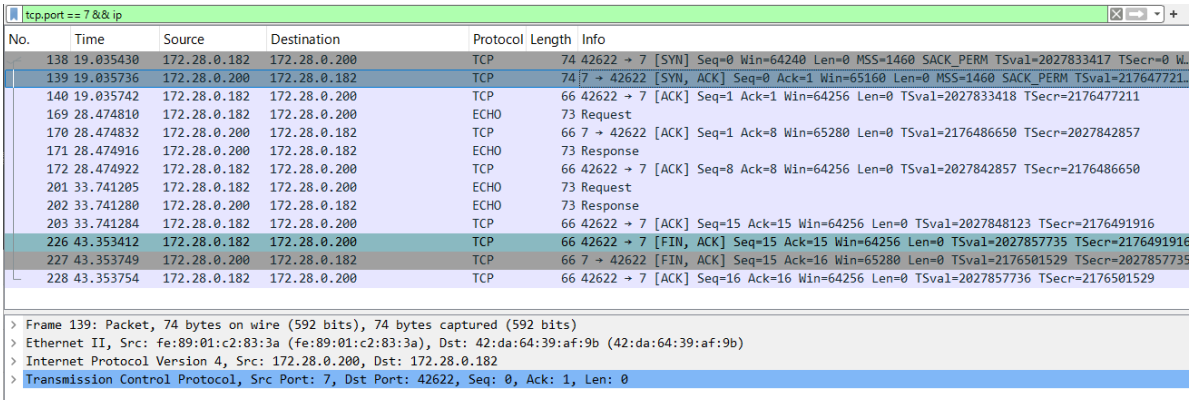



図 5: IPv4 の TCP echo 通信を Wireshark で読み取った結果

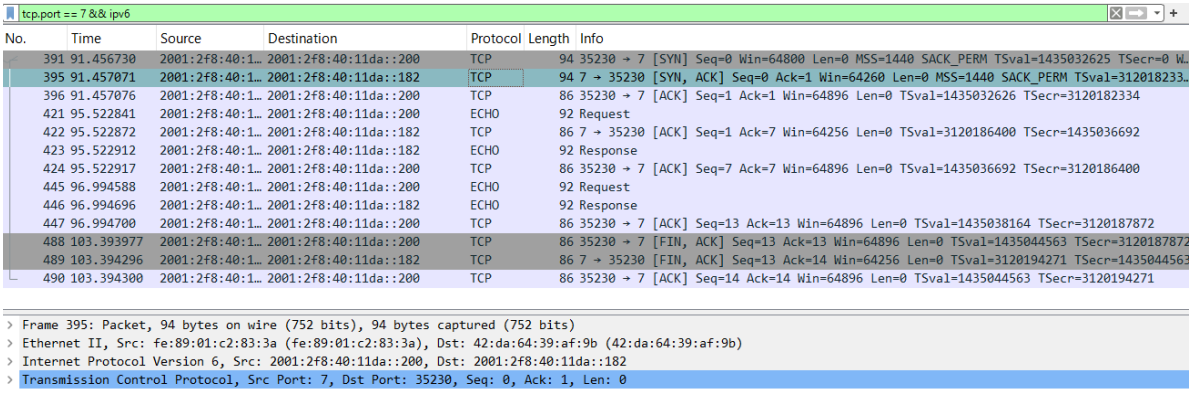



図 6: IPv6 の TCP echo 通信を Wireshark で読み取った結果

IPv4 では，出力 11 のとおり，入力した hello と world がそれぞれそのままサーバから返ってきた．図 5 では，まず No.138 の SYN，No.139 の SYN/ACK，No.140 の ACK により，TCP の 3 ウェイハンドシェイクで接続が確立している．続いて，No.169 で hello を含む ECHO の Request（データ長 7 バイト）がクライアント (172.28.0.182) からサーバ (172.28.0.200) へ送られ，サーバは No.170 で ACK を返したのち，No.171 で同じ 7 バイトの Response を返している．これをクライアントが No.172 で ACK している．world についても同様に，No.201 の Request，No.202 の Response，No.203 の ACK の順にやり取りされている．データ長が 7 バイトであるのは，hello の 5 バイトに，Enter で付加される改行コード (CR LF) の 2 バイトが加わっているためと考えられる．最後に，クライアントが quit で接続を終了した際，No.226 の FIN/ACK，No.227 の FIN/ACK，No.228 の ACK により接続が閉じられている．

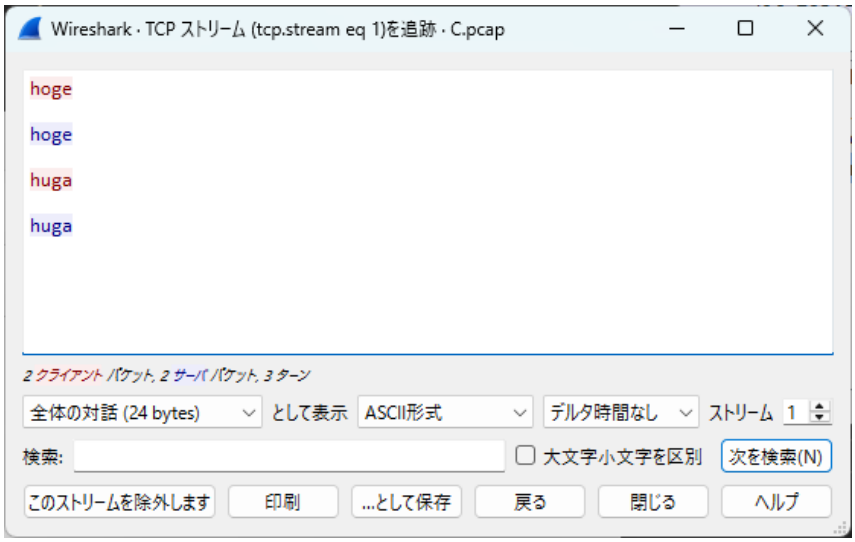



図 7: TCP ストリームの追跡結果 (IPv6)

IPv6 でも，出力 12 のとおり，入力した hoge と huga がそのまま返ってきた．図 6 では，クライアント (2001:2f8:40:11da::182) とサーバ (2001:2f8:40:11da::200) の間で，IPv4 と同じ手順で通信が行われている．No.391 の SYN，No.395 の SYN/ACK，No.396 の ACK で接続が確立し，No.421 の Request に対して No.422 の ACK，No.423 の Response（データはいずれも 6 バイト），No.424 の ACK が続く．huga についても，No.445 の Request，No.446 の Response，No.447 の ACK の順にやり取りされている．接続の終了は，No.488 の FIN/ACK，No.489 の FIN/ACK，No.490 の ACK である．図 7 の TCP ストリームの追跡 (IPv6) では，hoge と huga が送信側・受信側の双方で一致して表示され，全体の対話は 24 バイトであった．これは，6 バイトのデータが 4 回（送信 2 回，受信 2 回）やり取りされた合計と一致する．

IPv4 と IPv6 を比べると，TCP の手順（3 ウェイハンドシェイク，データの送受信，FIN による終了）は同じである．一方で，SYN パケットの長さは IPv4 の 74 バイトに対して IPv6 では 94 バイトと 20 バイト大きい．これは IP ヘッダが 20 バイトから 40 バイトになったためである．また，SYN に含まれる MSS の値も，IPv4 の 1460 に対して IPv6 では 1440 と 20 バイト小さく，同じ理由で説明できる．なお，echo サービスのポート番号は 7 であり，いずれの通信もサーバ側のポートは 7 である．

## 課題 2-D

### 課題内容

LAN 内の iperf3 サーバに対し，低速の UDP 通信を順方向・逆方向 (-R) の双方で行い，スループットとパケットの流れを確認した．

### 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w day2_D.pcap 'not port 4200'
```

2. キャプチャ中に，別のターミナルで iperf3 を順方向，逆方向の順に実行

```
（順方向）iperf3 -t 5 -u -b 10k -c 2001:2f8:40:11da::200
（逆方向）iperf3 -t 5 -R -u -b 10k -c 2001:2f8:40:11da::200
```

3. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開き，フィルタ `udp.port == 5201` を適用

## 結果

tcpdump の実行結果と，順方向・逆方向の iperf3 の実行結果をそれぞれ出力 13，出力 14，出力 15 に示した．

出力 13: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_D.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C520 packets captured
522 packets received by filter
0 packets dropped by kernel
```

出力 14: iperf3(順方向) の実行結果

```
guest@host82:~$ iperf3 -t 5 -u -b 10k -c 2001:2f8:40:11da::200
Connecting to host 2001:2f8:40:11da::200, port 5201
[  5] local 2001:2f8:40:11da::182 port 39751 connected to 2001:2f8:40:11da::200 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   1.00-2.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   2.00-3.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   3.00-4.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   4.00-5.00   sec  1.39 KBytes  11.4 Kbits/sec  1
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   sec  6.97 KBytes  11.4 Kbits/sec  0.000 ms  0/5 (0%)  sender
[  5]   0.00-5.04   sec  6.97 KBytes  11.3 Kbits/sec  0.003 ms  0/5 (0%)  receiver

iperf Done.
```

出力 15: iperf3(逆方向) の実行結果

```
guest@host82:~$ iperf3 -t 5 -R -u -b 10k -c 2001:2f8:40:11da::200
Connecting to host 2001:2f8:40:11da::200, port 5201
Reverse mode, remote host 2001:2f8:40:11da::200 is sending
[  5] local 2001:2f8:40:11da::182 port 37219 connected to 2001:2f8:40:11da::200 port 5201
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   sec  1.39 KBytes  11.4 Kbits/sec  2.541 ms  0/1 (0%)
[  5]   1.00-2.00   sec  1.39 KBytes  11.4 Kbits/sec  4.921 ms  0/1 (0%)
[  5]   2.00-3.00   sec  1.39 KBytes  11.4 Kbits/sec  4.613 ms  0/1 (0%)
[  5]   3.00-4.00   sec  1.39 KBytes  11.4 Kbits/sec  4.325 ms  0/1 (0%)
[  5]   4.00-5.00   sec  1.39 KBytes  11.4 Kbits/sec  4.055 ms  0/1 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-5.04   sec  6.97 KBytes  11.3 Kbits/sec  0.000 ms  0/5 (0%)  sender
[  5]   0.00-5.00   sec  6.97 KBytes  11.4 Kbits/sec  4.055 ms  0/5 (0%)  receiver

iperf Done.
```

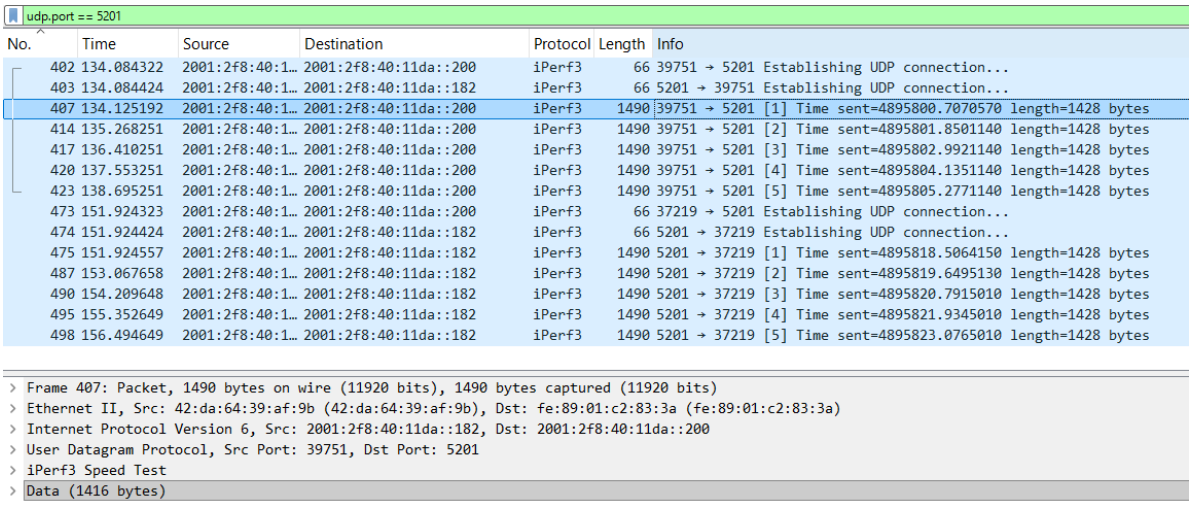



図 8: 順方向の UDP 通信を Wireshark で読み取った結果

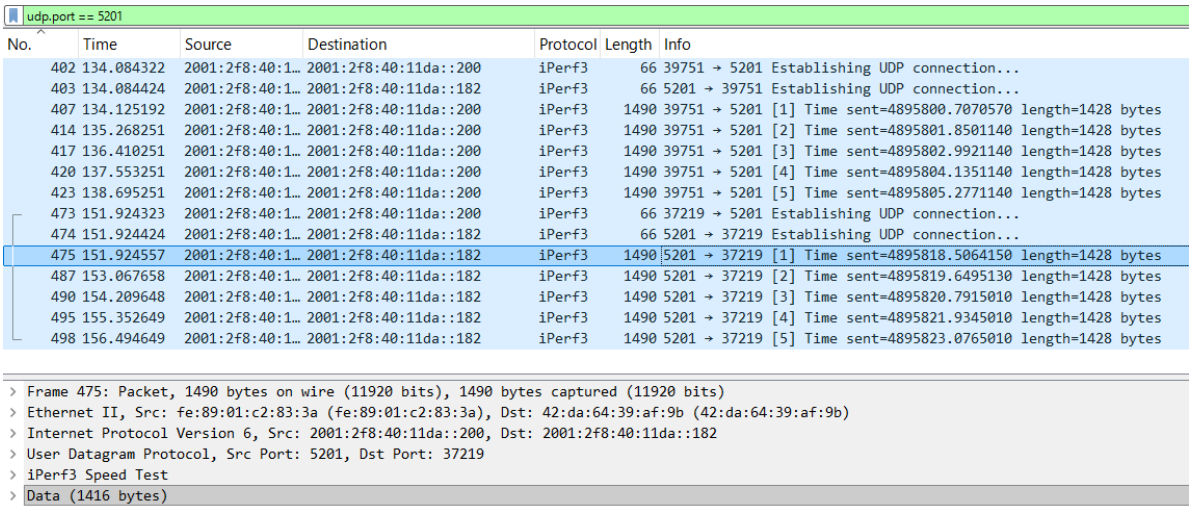



図 9: 逆方向の UDP 通信を Wireshark で読み取った結果

順方向では, 出力 14 のとおり, 指定した 10 kbit/s に対し, 送信側の実測 Bitrate は 11.4 kbit/s であった. 5 個

のデータグラムが送信され, 損失は 0 個であった. 図 8 では, No.402 でクライアント (2001:2f8:40:11da::182) からサーバ (2001:2f8:40:11da::200) へ UDP 5201 番ポートの接続手続き用パケットが送られ，No.403 でサーバから応答が返っている．続いて，No.407，414，417，420，423 でデータグラムが送られている．各パケットの UDP ペイロード長は 1428 バイトであり，送信間隔は約 1.142 秒であった．$L = 1428$ バイトとすると，指定した 10 kbit/s から計算される送信間隔は $8L/10000 = 1.1424$ 秒となり，観測値とほぼ一致する．

逆方向 (-R) では，出力 15 のとおり，受信側の Bitrate は 11.4 kbit/s であり，5 個のデータグラムの損失は 0 個であった．図 9 では，No.473，474 で接続手続きが行われた後，No.475，487，490，495，498 でサーバ (2001:2f8:40:11da::200) からクライアント (2001:2f8:40:11da::182) へデータグラムが送られている．順方向とはデータグラムの送信方向が逆になっていることが確認できた．

なお，表示フィルタ `udp.port == 5201` では，iperf3 の制御に使われる TCP 接続は表示されない．このため，図 8 および図 9 では，UDP による接続手続きとデータグラムの流れを確認した．

## 課題 2-E

### 課題内容

学外の Web サイトに wget で HTTP/HTTPS アクセスし，通信の手順（ゲートウェイの MAC アドレス解決，DNS，HTTP，HTTPS）を確認した．

### 実験方法

1. 経路表と近隣キャッシュを記録

```
ip -4 route
ip -6 route
ip ne
```

2. 以下のサイトごとに，キャプチャ開始，wget 実行，キャプチャ停止を繰り返す（pcap はサイトごとに分ける）．tcpdump と wget は別のターミナルで実行し，wget の終了後に Ctrl-C でキャプチャを止める．

```
(IPv4 のみ)     tcpdump -i eth1 -s 0 -w day2_E_sapporo.pcap 'not port 4200'
                wget www.city.sapporo.jp
(IPv4/IPv6)     tcpdump -i eth1 -s 0 -w day2_E_hokkaido.pcap 'not port 4200'
                wget www.pref.hokkaido.lg.jp
(IPv6 のみ指定) tcpdump -i eth1 -s 0 -w day2_E_keio.pcap 'not port 4200'
                wget www.keio.ac.jp
```

3. 各 pcap を Wireshark で開き，次のフィルタを適用

```
(arp || icmpv6.type == 135 || icmpv6.type == 136 || dns || tcp.port == 80 || tcp.port == 443) &&
↪  !(dns.qry.type == 12)
```

## 結果

実行前の経路表と近隣キャッシュを出力 16 に，サイトごとの tcpdump と wget の実行結果を出力 17，出力 18，出力 19 に示した．

出力 16: 実行前の経路表と近隣キャッシュ

```
guest@host82:~$ ip -4 route
default via 172.28.0.1 dev eth1
172.27.0.0/24 dev eth0 proto kernel scope link src 172.27.0.182
172.28.0.0/24 dev eth1 proto kernel scope link src 172.28.0.182

guest@host82:~$ ip -6 route
2001:2f8:40:11da::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
default via 2001:2f8:40:11da::1 dev eth1 metric 1024 pref medium

guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::fc89:1ff:fec2:833a dev eth1 lladdr fe:89:01:c2:83:3a STALE
2001:2f8:40:11da::200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
2001:2f8:40:11da::203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed REACHABLE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
```

出力 17: www.city.sapporo.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_sapporo.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C152 packets captured
154 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.city.sapporo.jp
--2026-11-10 00:55:19--  http://www.city.sapporo.jp/
www.city.sapporo.jp (www.city.sapporo.jp) を DNS に問いあわせています... 45.60.112.77
www.city.sapporo.jp (www.city.sapporo.jp)|45.60.112.77|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: https://www.city.sapporo.jp/ [続く]
--2026-11-10 00:55:19--  https://www.city.sapporo.jp/
www.city.sapporo.jp (www.city.sapporo.jp)|45.60.112.77|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 38349 (37K) [text/html]
`index.html' に保存中

index.html                100%[=====================================>]  37.45K  --.-KB/s    in 0.03s

2026-11-10 00:55:20 (1.27 MB/s) - `index.html' へ保存完了 [38349/38349]
```

出力 18: www.pref.hokkaido.lg.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_hokkaido.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C170 packets captured
172 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.pref.hokkaido.lg.jp
--2026-11-10 00:57:19--  http://www.pref.hokkaido.lg.jp/
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp) を DNS に問いあわせています... 2a02:e980:10e::4d,
↪  45.60.112.77
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp)|2a02:e980:10e::4d|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 302 Found
場所: https://www.pref.hokkaido.lg.jp/ [続く]
--2026-11-10 00:57:19--  https://www.pref.hokkaido.lg.jp/
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp)|2a02:e980:10e::4d|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 特定できません [text/html]
`index.html.1' に保存中

index.html.1                    [ <=>                                   ]  93.46K  --.-KB/s    in 0.07s

2026-11-10 00:57:20 (1.40 MB/s) - `index.html.1' へ保存終了 [95704]
```

出力 19: www.keio.ac.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_keio.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C296 packets captured
298 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.keio.ac.jp
--2026-11-10 00:59:01--  http://www.keio.ac.jp/
www.keio.ac.jp (www.keio.ac.jp) を DNS に問いあわせています... 2600:9000:2142:5800:9:dcf4:5ec0:93a1,
↪  2600:9000:2142:50
00:9:dcf4:5ec0:93a1, 2600:9000:2142:3c00:9:dcf4:5ec0:93a1, ...
www.keio.ac.jp (www.keio.ac.jp)|2600:9000:2142:5800:9:dcf4:5ec0:93a1|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: https://www.keio.ac.jp/ [続く]
--2026-11-10 00:59:01--  https://www.keio.ac.jp/
www.keio.ac.jp (www.keio.ac.jp)|2600:9000:2142:5800:9:dcf4:5ec0:93a1|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: /ja/ [続く]
--2026-11-10 00:59:01--  https://www.keio.ac.jp/ja/
[www.keio.ac.jp]:443 への接続を再利用します。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 372926 (364K) [text/html]
`index.html.2' に保存中

index.html.2              100%[=====================================>] 364.19K  --.-KB/s    in 0.05s

2026-11-10 00:59:01 (6.60 MB/s) - `index.html.2' へ保存完了 [372926/372926]
```

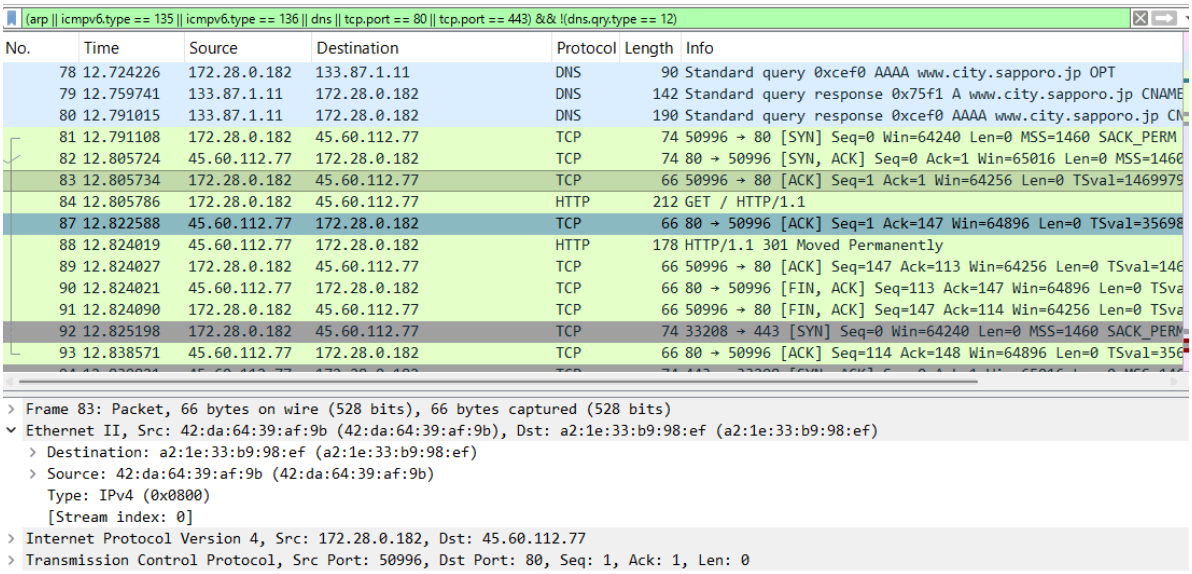



図 10: www.city.sapporo.jp へのアクセスを Wireshark で読み取った結果

経路表では，デフォルトゲートウェイは IPv4 が 172.28.0.1，IPv6 が 2001:2f8:40:11da::1 であり，実行前の近隣キャッシュにはどちらも MAC アドレス `a2:1e:33:b9:98:ef` が登録されていた．そのため，クライアントがゲートウェイの MAC アドレスを新たに問い合わせる ARP や ND は確認されなかった．図 10 で

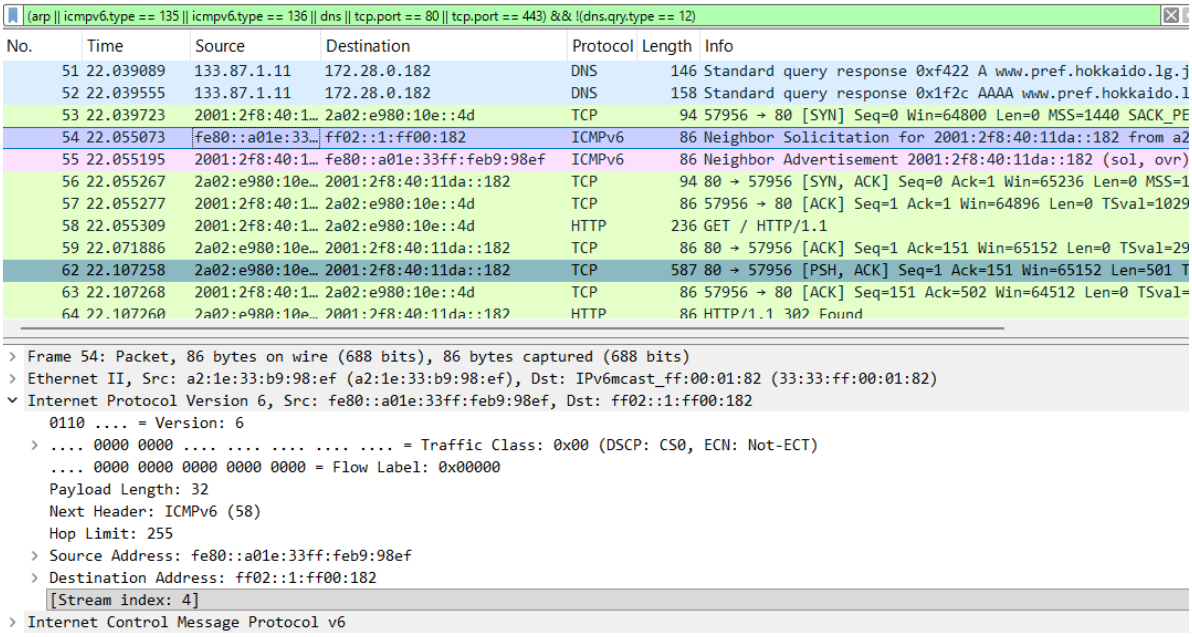



図 11: www.pref.hokkaido.lg.jp へのアクセスを Wireshark で読み取った結果

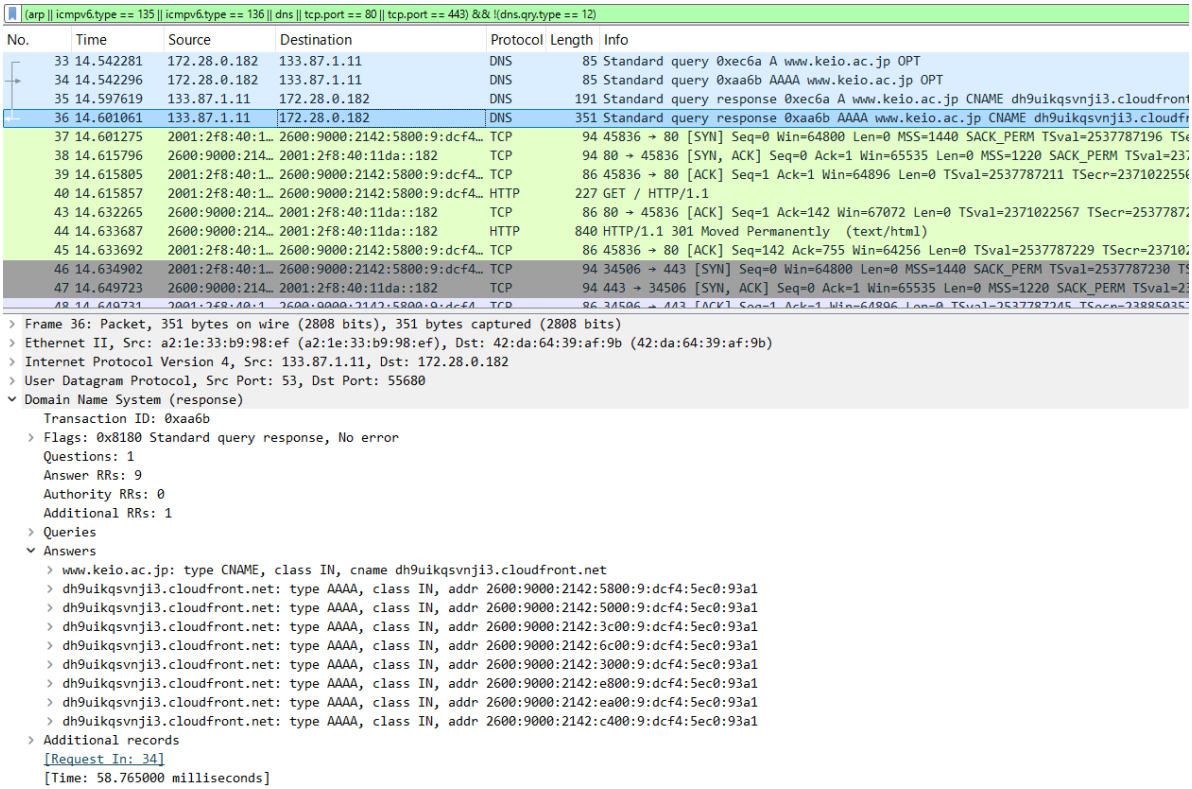



図 12: www.keio.ac.jp へのアクセスを Wireshark で読み取った結果

は，No.83 の Ethernet II ヘッダに，学外の宛先へ送るフレームの宛先 MAC アドレスとしてゲートウェイの MAC アドレスが表示されている．

www.city.sapporo.jp では, No.77 で A レコード, No.78 で AAAA レコードが DNS リゾルバ 133.87.1.11 に問い合わせられた．No.79 の応答では，CNAME`i4tz2oz.impervadns.net` を経由して IPv4 アドレス 45.60.112.77 が返された．No.80 の AAAA 応答には IPv6 アドレスがなく，CNAME と SOA が返されたため，このキャプチャでは IPv4 のみで接続可能であることが分かる．No.84 で HTTP GET が送られ，No.88 では 301 Moved Permanently と Location ヘッダによって HTTPS へ転送された．その後，ポート 443 で TLS のハンドシェイクが行われ，HTTPS でコンテンツを取得した．

www.pref.hokkaido.lg.jp では，No.49 と No.50 で A および AAAA レコードが問い合わせられ，No.51 と No.52 の応答で IPv4 アドレス 45.60.112.77 と IPv6 アドレス 2a02:e980:10e::4d が返された．TCP SYN の宛先は IPv6 アドレスであり，wget は IPv6 を使って接続した．No.58 で HTTP GET が送られ，No.64 で 302 Found と Location ヘッダによって HTTPS へ転送された後，TLS のハンドシェイクを経て HTTPS でコンテンツを取得した．

www.keio.ac.jp では，No.33 と No.34 で A および AAAA レコードが問い合わせられ，No.35 と No.36 の応答で IPv4 と複数の IPv6 アドレスが返された．TCP SYN の宛先は IPv6 アドレス 2600:9000:2142:5800:9:dcf4:5ec0:93a1 であり，wget は IPv6 を使って接続した．No.41 で HTTP GET が送られ，No.44 で 301 Moved Permanently により HTTPS へ転送された．さらに，wget の出力では HTTPS 接続後に/ja/へ転送され，最終的に 200 OK の応答を得ている．

以上より，札幌市のサイトでは AAAA レコードが返らず，IPv4 で接続していたのに対し，北海道庁と慶應義塾大学のサイトでは A と AAAA の両方が返り，wget は IPv6 で接続していた．3 サイトとも HTTP から HTTPS へのリダイレクトがあり，サーバから返されるステータスコードや，追加のリダイレクト先には違いが見られた．

## 課題 3-1

### 課題内容

Python の DNS リゾルバ (dnspython) で，IPv4 のみ対応のホストと IPv4/IPv6 両対応のホストの A・AAAA レコードを引き，その通信を分析した．

### 実験方法

1. ひな型を自分のホームにコピー

```
cd; cp -r /shares/dns .; chmod -R u+w dns; cd dns
```

2. ホスト名ごとに，キャプチャ開始，q_hostname の書き換え，実行，キャプチャ停止を行う

```
(IPv4 のみ)  tcpdump -i eth1 -s 0 -w day3_1_v4.pcap 'not port 4200'    (1 つ目のターミナル)
             sed -i "s/^q_hostname=.*/q_hostname='www.city.sapporo.jp'/" dnspython_sample.py
             ./dnspython_sample.py
(両対応)     tcpdump -i eth1 -s 0 -w day3_1_v46.pcap 'not port 4200'   (1 つ目のターミナル)
             sed -i "s/^q_hostname=.*/q_hostname='www.pref.hokkaido.lg.jp'/" dnspython_sample.py
```

```
./dnspython_sample.py
```

3. 各 pcap を Wireshark で開き，次のフィルタを適用

```
udp.port == 53 && !(dns.qry.type == 12)
```

逆引き (PTR, Type 12) は本課題と無関係なので除外した．

## 結果

tcpdump の実行結果と，dnspython_sample.py の実行結果を出力 20，出力 21，出力 22，出力 23 に示した．使用したプログラムのソースコードをリスト 1 に示す．`q_hostname` の行は，ホスト名ごとに `sed` で書き換えて使用した．

```
#!/usr/bin/python3

import dns.resolver

#q_hostname='google.co.jp'
q_hostname='www.city.sapporo.jp'
#q_hostname='www.yomiuri.co.jp'

print('Querying hostname:', q_hostname)

# DNS問い合わせ方式の選択
# 'normal' : 通常DNS（Do53、OSのデフォルトリゾルバ）
# 'doh'    : DNS over HTTPS
# 'dot'    : DNS over TLS

mode = 'normal'

if mode == 'doh':
    # Cloudflare Public DoH Resolver
    resolver = dns.resolver.Resolver(configure=False)
    resolver.nameservers = [
        dns.nameserver.DoHNameserver(
            'https://cloudflare-dns.com/dns-query',
            bootstrap_address='1.1.1.1'
        )
    ]

elif mode == 'dot':
    # Cloudflare Public DoT Resolver
    resolver = dns.resolver.Resolver(configure=False)
    resolver.nameservers = [
        dns.nameserver.DoTNameserver(
            '1.1.1.1',
            port=853,
            hostname='one.one.one.one',
            verify=True
        )
    ]

elif mode == 'normal':
    # /etc/resolv.conf のデフォルトリゾルバ
    resolver = dns.resolver.Resolver()

else:
    raise ValueError('Unknown DNS mode: ' + mode)

resolver.timeout = 5
resolver.lifetime = 15

try:
  answer1 = resolver.resolve(q_hostname, 'A')
```

出力 20: tcpdump(IPv4 のみのホスト) の実行結果

```
guest@host82:~/dns$ tcpdump -i eth1 -s 0 -w day3_1_v4.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C58 packets captured
60 packets received by filter
0 packets dropped by kernel
```

出力 21: tcpdump(両対応のホスト) の実行結果

```
guest@host82:~/dns$ tcpdump -i eth1 -s 0 -w day3_1_v46.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C22 packets captured
24 packets received by filter
0 packets dropped by kernel
```

出力 22: dnspython_sample.py(www.city.sapporo.jp) の実行結果

```
guest@host82:~/dns$ ./dnspython_sample.py
Querying hostname: www.city.sapporo.jp
A records are:
  45.60.112.77
  Replied nameserver: 133.87.1.11 (53)
AAAA records: No answer from the resolver.
```

出力 23: dnspython_sample.py(www.pref.hokkaido.lg.jp) の実行結果

```
guest@host82:~/dns$ ./dnspython_sample.py
Querying hostname: www.pref.hokkaido.lg.jp
A records are:
  45.60.112.77
  Replied nameserver: 133.87.1.11 (53)
AAAA records are:
  2a02:e980:10e::4d
  Replied nameserver: 133.87.1.11 (53)
```

IPv4 のみのホスト (www.city.sapporo.jp) では，図 13 のとおり，No.69 で A レコード，No.71 で AAAA レコードの問い合わせが，リゾルバ 133.87.1.11(UDP 53 番ポート) に送られた．図 14 では，A の応答 (No.70) には，CNAME(i4tz2oz.impervadns.net) を経由してアドレス 45.60.112.77(TTL 119 秒) が含まれている一方，AAAA の応答 (No.74) には CNAME と SOA のみが含まれ，AAAA レコードはなかった．これは，このホストが IPv6 のアドレスを持たないことを示す．出力 22 でも，A レコードとして 45.60.112.77 が 1 件表示された一方，AAAA は “No answer from the resolver” と表示されており，パケットの内容と一致する．

両対応のホスト (www.pref.hokkaido.lg.jp) では，図 15 のとおり，No.95 で A レコード，No.98 で AAAA レコードの問い合わせが同様にリゾルバ 133.87.1.11 に送られた．図 16 では，A の応答 (No.96) に IPv4 アドレス 45.60.112.77(TTL 46 秒)，AAAA の応答 (No.100) に IPv6 アドレス 2a02:e980:10e::4d(TTL 46 秒) が含まれた．どちらの応答にも，www.pref.hokkaido.lg.jp から wgobu9h.impervadns.net への CNAME が含まれ，その先の A レコード，AAAA レコードとしてアドレスが得られている．出力 23 にも，同じ 2 つの

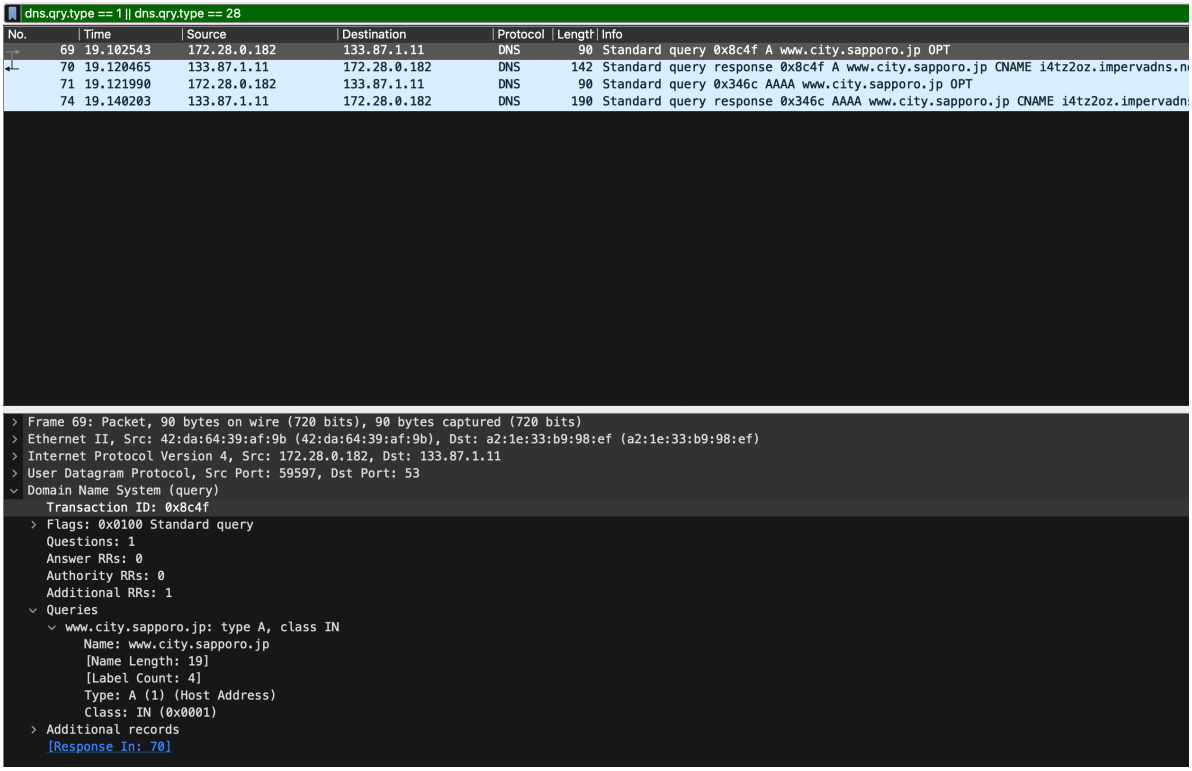



図 13: IPv4 のみのホストの A・AAAA の問い合わせ

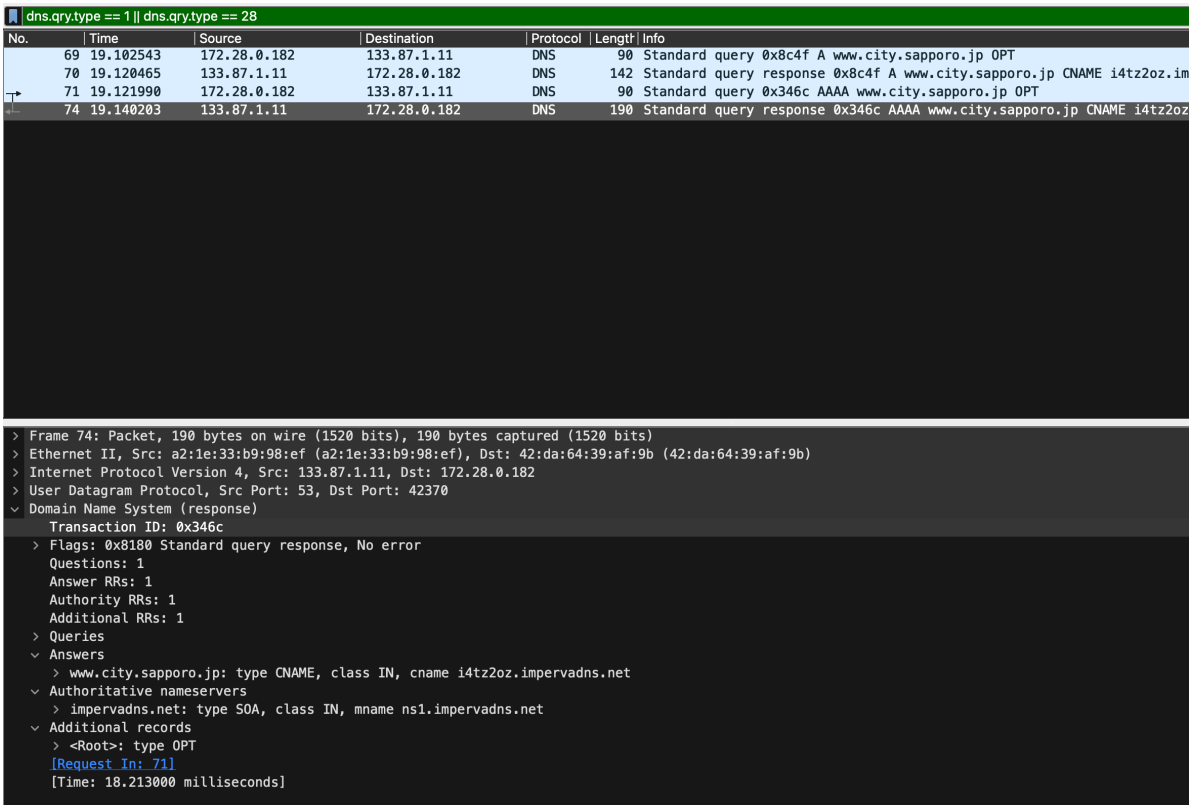



図 14: IPv4 のみのホストの A・AAAA の応答

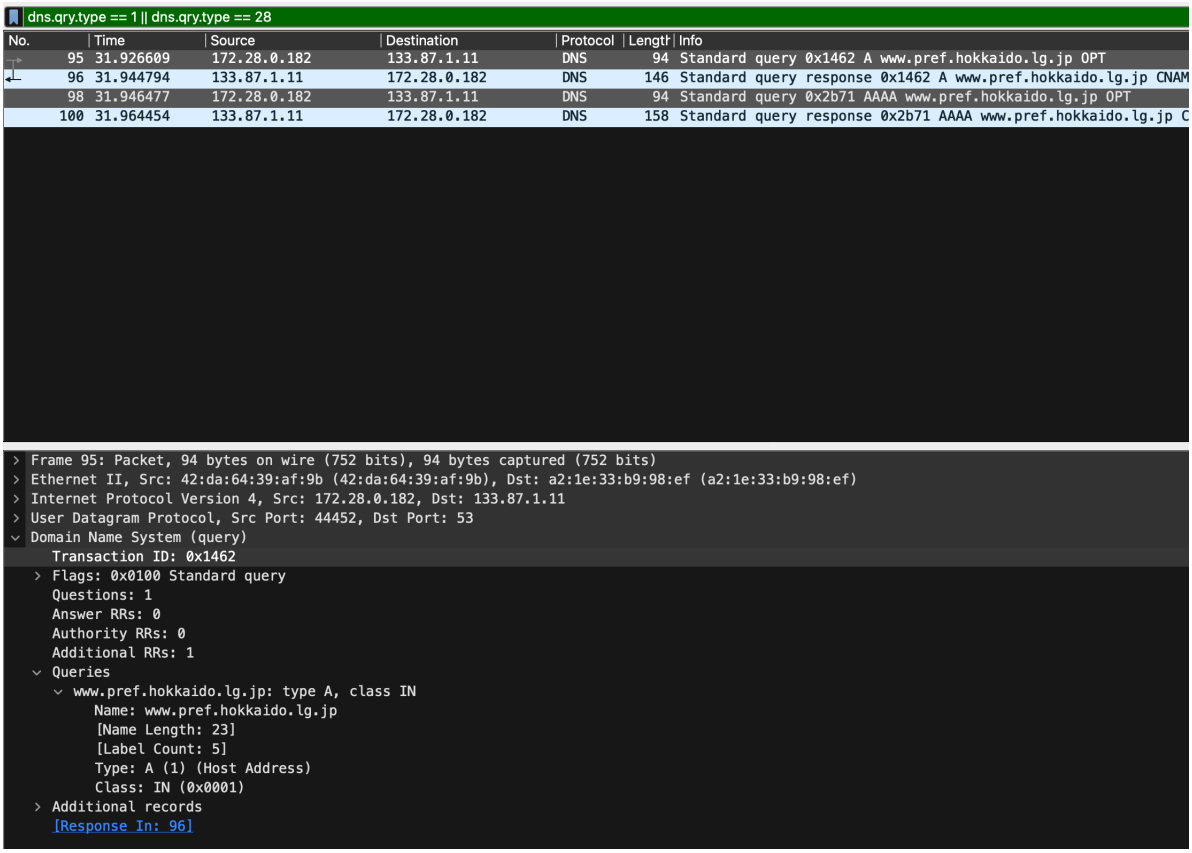



図 15: 両対応のホストの A・AAAA の問い合わせ

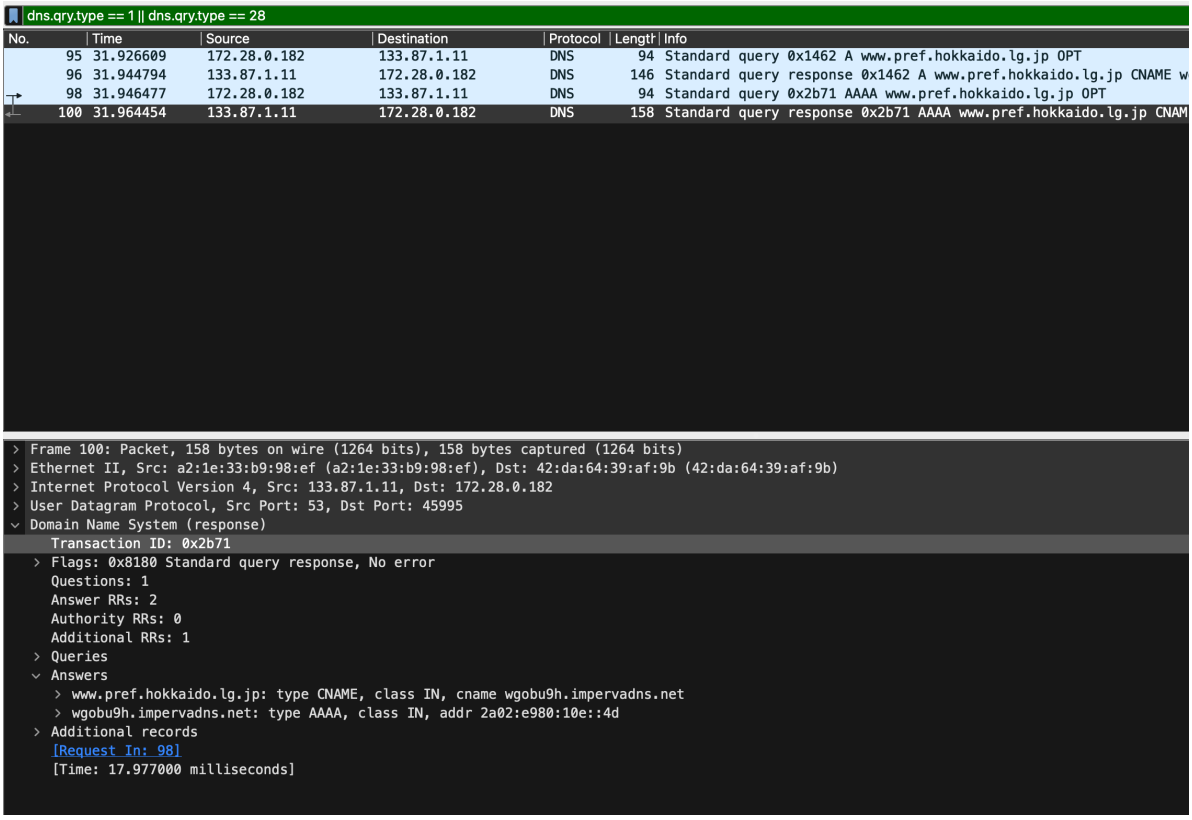



図 16: 両対応のホストの A・AAAA の応答

アドレスが表示されている．

2 つのホストを比べると，どちらでも A と AAAA の 2 種類の問い合わせが同じリゾルバ (133.87.1.11) に送られており，問い合わせの手順は同じである．違いは応答の内容にあり，IPv4 のみのホストでは AAAA の応答にアドレスがなく，両対応のホストでは両方のアドレスが得られた．また，2 つのホストの A レコードは，どちらも 45.60.112.77 と同じ値であった．これは，どちらのホストも impervadns.net 配下の名前への CNAME を経由しており (sapporo は i4tz2oz.impervadns.net，hokkaido は wgobu9h.impervadns.net)，同じ事業者のサービス上に置かれているためと考えられる．また，A と AAAA の問い合わせは並行ではなく順に送られていた．A の応答を受け取った約 1.5 ミリ秒後 (No.70 と No.71 の間) に AAAA の問い合わせが送られており，両対応のホストでも同様に，A の応答 (No.96) の約 1.7 ミリ秒後に AAAA の問い合わせ (No.98) が送られている．

## 課題 3-2

### 課題内容

socket の getaddrinfo で名前解決し，IPv4 のみ対応のホストと両対応のホストで返るアドレスの違いを確認した．

### 実験方法

1. ホスト名を書き換えて実行（3-1 と同じ 2 つのホスト）

```
sed -i "s/^q_hostname = .*/q_hostname = 'www.city.sapporo.jp'/" dns_socket_sample.py
./dns_socket_sample.py
sed -i "s/^q_hostname = .*/q_hostname = 'www.pref.hokkaido.lg.jp'/" dns_socket_sample.py
./dns_socket_sample.py
```

   出力に `AF_INET` のみが並ぶか，`AF_INET6` も並ぶかを比較した．

### 結果

2 つのホストに対する実行結果を，出力 24，出力 25 に示した．使用したプログラムのソースコードをリスト 2 に示す．`q_hostname` の行は，ホスト名ごとに `sed` で書き換えて使用した．

```python
#!/usr/bin/python3

import socket

q_hostname = 'www.pref.hokkaido.lg.jp'

print('Quering hostname:', q_hostname)

addrs = socket.getaddrinfo(q_hostname, 443)
for addr in addrs:
    print(addr)
```

ソースコード 2: dns_socket_sample.py のソースコード

出力 24: dns_socket_sample.py(www.city.sapporo.jp) の実行結果

```
guest@host82:~/dns$ ./dns_socket_sample.py
Quering hostname: www.city.sapporo.jp
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_STREAM: 1>, 6,
 '', ('45.60.112.77', 443))
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_DGRAM: 2>, 17,
 '', ('45.60.112.77', 443))
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_RAW: 3>, 0,
 '', ('45.60.112.77', 443))
```

出力 25: dns_socket_sample.py(www.pref.hokkaido.lg.jp) の実行結果

```
guest@host82:~/dns$ ./dns_socket_sample.py
Quering hostname: www.pref.hokkaido.lg.jp
(<AddressFamily.AF_INET6: 10>, <SocketKind.SOCK_STREAM: 1>, 6,
 '', ('2a02:e980:10e::4d', 443, 0, 0))
(<AddressFamily.AF_INET6: 10>, <SocketKind.SOCK_DGRAM: 2>, 17,
 '', ('2a02:e980:10e::4d', 443, 0, 0))
(<AddressFamily.AF_INET6: 10>, <SocketKind.SOCK_RAW: 3>, 0,
 '', ('2a02:e980:10e::4d', 443, 0, 0))
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_STREAM: 1>, 6,
 '', ('45.60.112.77', 443))
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_DGRAM: 2>, 17,
 '', ('45.60.112.77', 443))
(<AddressFamily.AF_INET: 2>, <SocketKind.SOCK_RAW: 3>, 0,
 '', ('45.60.112.77', 443))
```

www.city.sapporo.jp では，出力 24 のとおり，返ってきたアドレスはすべて AF_INET で，IPv4 アドレス 45.60.112.77(ポート 443) だけが得られた．同じアドレスが 3 行並んでいるのは，ソケットの種類 (SOCK_STREAM，SOCK_DGRAM，SOCK_RAW) ごとに 1 つずつ返されたためで，プロトコル番号はそれぞれ 6(TCP)，17(UDP)，0 である．これは 3-1 で，このホストの AAAA の応答にアドレスがなかったことと一致する．

www.pref.hokkaido.lg.jp では, 出力 25 のとおり, AF_INET6 のアドレス 2a02:e980:10e::4d と, AF_INET

のアドレス 45.60.112.77 の両方が返った．各アドレスがソケットの種類ごとに 3 行ずつ並ぶ点は同じで，合計 6 行であった．また，AF_INET6 が先に並び，AF_INET が後に続いた．getaddrinfo は RFC 6724 に基づいて宛先アドレスを並べ替えるため，IPv6 の優先度が IPv4 より高く，IPv6 が先頭になったと考えられる．実際，課題 E の wget でも，www.pref.hokkaido.lg.jp への接続には IPv6 が使われており，この順序と一致する．

以上より，getaddrinfo は，DNS の A と AAAA の結果をまとめて返し，呼び出し側はそのアドレスファミリを見て接続先を選べる．ホストが IPv6 に対応しているかどうかは，返るアドレスの種類で判断できる．

## 課題 4-1

### 課題内容

UDP サーバ udp_server.py と UDP クライアント udp_client.py を実行し，クライアントから送信したデータをサーバが受信することを確認する．

### 実験方法

1. サンプルをホームディレクトリにコピーし，サーバを起動する．

```
cd
cp -r /shares/socket .
cd socket
./udp_server.py
```

2. 別の演習環境からクライアントを起動する．

```
cd socket
./udp_client.py
```

3. サーバ側とクライアント側の出力を確認し，サーバを Ctrl-C で停止する．

### 結果

サーバ側とクライアント側の実行結果を出力 26，出力 27 に示す．使用したソースコードをリスト 3，リスト 4 に示す．

```python
#!/usr/bin/python3

import socket
from datetime import datetime

# 特定のクライアントからの通信のみを許可したい場合は '' の箇所にそのホスト名またはアドレスを記載
HOSTNAME = ''
PORT_NUM = 6789

# 接続するサーバのポート番号を指定
MAX_SIZE = 4096

print('Starting the server at', datetime.now())
print('Waiting for a client to call.')


# 1. socket の作成
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. bind 手続き（socket に通信相手の IP アドレス、ポート番号の指定）
server.bind((HOSTNAME, PORT_NUM))

# 3. socket からデータの受信
data, client = server.recvfrom(MAX_SIZE)

print('At', datetime.now(), client, 'said,', data.decode('utf-8'))

# 4. socket に対するデータの送信
server.sendto(b'Are you talking to me?', client)

# 5. socket の閉鎖
server.close()
```

ソースコード 3: udp_server.py のソースコード

```python
#!/usr/bin/python3

import socket
from datetime import datetime

HOSTNAME = 'localhost'
PORT_NUM = 6789

MAX_SIZE= 1024

print('Staring the client at', datetime.now())

# 1. socket の作成 (AF_INET は IPv4 インターネット, SOCK_DGRAM はデータグラム型 (UDP)
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. socket に対するデータの送信
client.sendto(b'Hey!', (HOSTNAME, PORT_NUM))

# 3. socket からデータの受信
data, server = client.recvfrom(MAX_SIZE)

print('At', datetime.now(), server, 'said,', data.decode('utf-8'))

# 4. socket の閉鎖
client.close()
```

ソースコード 4: udp_client.py のソースコード

出力 26: udp_server.py の実行結果

```
guest@host82:~/socket$ ./udp_server.py
Starting the server at 2026-11-12 00:27:30.085333
Waiting for a client to call.
At 2026-11-12 00:28:28.445504 ('127.0.0.1', 40714) said, Hey!
```

出力 27: udp_client.py の実行結果

```
guest@host82:~$ cd ~/socket
guest@host82:~/socket$ ./udp_client.py
Staring the client at 2026-11-12 00:28:28.445191
At 2026-11-12 00:28:28.445576 ('127.0.0.1', 6789) said, Are you talking to me?
```

クライアントが送信した Hey! をサーバが受信し，サーバからの応答 Are you talking to me? をクライアントが受信した．UDP による要求と応答が正常に行われた．

# 課題 4-2

## 課題内容

`socket_dns_test.py` が DNS 問い合わせに設定した乱数の Query ID と，応答パケットの Response ID が一致することを確認する．

## 実験方法

1. DNS 問い合わせのパケットをキャプチャする．

```
cd ~/socket
tcpdump -i eth1 -s 0 -w day4_2_dns.pcap 'not port 4200'
```

2. 別の端末でプログラムを実行する．引数には実際に問い合わせたホスト名を指定する．

```
cd ~/socket
./socket_dns_test.py www.iic.hokudai.ac.jp
```

3. キャプチャを停止し，pcap を Wireshark で開いて DNS の Query と Response を確認する．

## 結果

プログラムの実行結果を出力 28 に示す．Wireshark で確認した Query ID と Response ID を図 17 に示す．使用した `socket_dns_test.py` のソースコードをリスト 5 に示す．

```
#!/usr/bin/python3

# coding=utf8
import socket
import struct
import random
import ipaddress

random.seed()

#DNSSERVER = '1.1.1.1'
DNSSERVER = '133.87.1.11'
PORT = 53


def resolve(fqdn):
  # DNS ゆーたら UDP でしょ
  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
  qdata = _query(fqdn)
  qid = qdata[:2]
  sock.sendto(qdata, (DNSSERVER, PORT))

  rdata = sock.recvfrom(1024)[0]
  # query の id をそのまま返してきてるはず
  rid = rdata[:2]

  # qr は response なら 1
  qr = rdata[3] >> 7
  # rcode は成功なら 0
  rcode = rdata[4] & 31

  # id 同じで response で成功なら parse する
  if (rid, qr, rcode) == (qid, 1, 0):
```

ソースコード 5: socket_dns_test.py のソースコード（1/4）

```
    return _parse(rdata)
  else:
    return None


def _query(fqdn):
  """fqdn の IP address を下さいという query を作る"""

  fqdn = fqdn.encode('utf-8')
  r = random.randint(0,2**16-1)
  print("id =", r)
  qid = struct.pack('>H', r)
  header = qid + b'\x01\x00\x00\x01\x00\x00\x00\x00\x00\x00'

  # d.hatena.ne.jp -> \x01d\x06hatena\x02ne\x02jp\x00
  qname = b''.join([chr(len(x)).encode() + x for x in fqdn.split(b'.')]) + b'\x00'
  # 1 が "a host address"
  qtype = struct.pack('>H', 1)

  # 28 が "aaaa IPv6 address"
  #qtype = struct.pack('>H', 28)

  # 1 が "the Internet"
  qclass = struct.pack('>H', 1)

  return header + qname + qtype + qclass


def _parse(data):
  """DNS response をパースして list of tuples で返す

  data -> [(name, type, class ttl, response_data)]
  """
```

ソースコード 6: socket_dns_test.py のソースコード（2/4）

```
  result = []
  i = 12 # これ hard coding でいいの?

  # response には query が含まれてるのね
  qname, i = _pick_name(data, i)
  qtype = struct.unpack('>H', data[i:i+2])[0]
  qclass = struct.unpack('>H', data[i+2:i+4])[0]
  i += 4

  while True:
    # response data を喰いつくす、全部読んだら終わり
    if len(data) <= i:
      break

    rec_name, i = _pick_name(data, i)
    rec_type = struct.unpack('>H', data[i:i+2])[0]
    rec_class = struct.unpack('>H', data[i+2:i+4])[0]
    rec_ttl = struct.unpack('>I', data[i+4:i+8])[0]
    rec_dlength = struct.unpack('>H', data[i+8:i+10])[0]

    if rec_type == 1:
      # A レコードだったら
      rec_data = '.'.join([str(x) for x in data[i+10:i+14]])
      i += 14
    elif rec_type == 5:
      # CNAME レコードだったら
      rec_data, i = _pick_name(data, i+10)
    elif rec_type == 28:
      # AAAA レコードだったら
      rec_data = str(ipaddress.IPv6Address(data[i+10:i+26]))
      i += 26
    else:
      # それ以外だったら parse しないで bytes として
```

ソースコード 7: socket_dns_test.py のソースコード（3/4）

```
      rec_data = data[i+10:i+10+rec_dlength]
      i = i+10+rec_dlength
    result.append((rec_name, rec_type, rec_class, rec_ttl, rec_data))

  return result


def _pick_name(data, num):
  """data の num bytes 目から始まる名前を拾う"""

  name_list = []
  while True:
    length = data[num]
    num += 1
    if length == 0:
      break
    elif length >= 192:
      # length (2 octets) の頭が 11 だと compressed なので別処理
      name_length = struct.unpack('>H', data[num-1:num+1])[0] % (3 << 14)
      name, _ = _pick_name(data, name_length)
      name_list.append(name)
      num += 1
      break
    else:
      name_list.append(data[num:num+length])
      num += length
  return b'.'.join(name_list), num


if __name__ == '__main__':
  import sys

  print(resolve(sys.argv[1]))
```

ソースコード 8: socket_dns_test.py のソースコード（4/4）

出力 28: socket_dns_test.py の実行結果

```
guest@host82:~/socket$ ./socket_dns_test.py www.iic.hokudai.ac.jp
id = 52921
[(b'www.iic.hokudai.ac.jp', 1, 1, 86400, '133.50.42.161')]
```

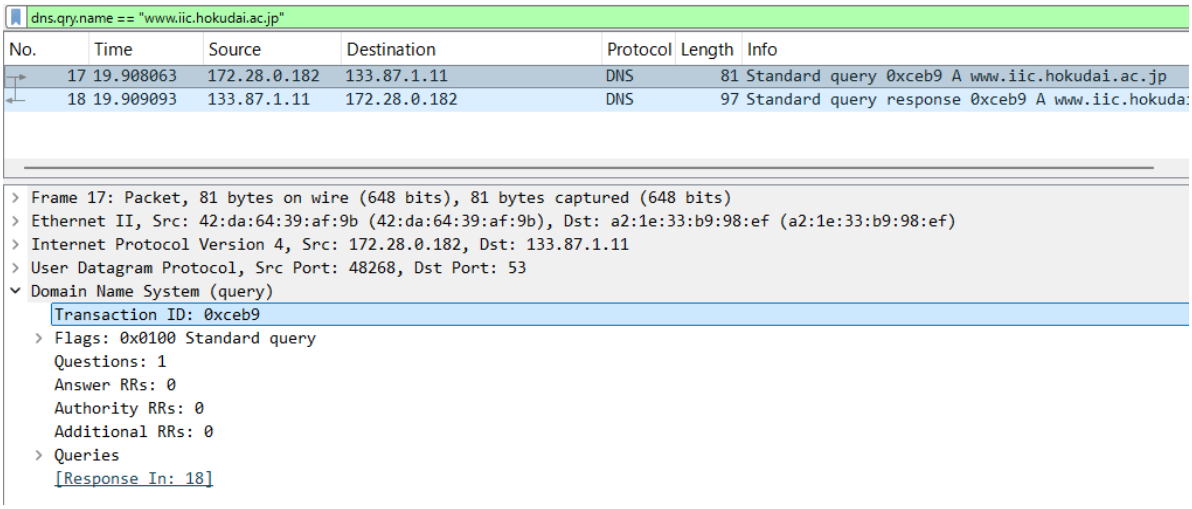



図 17: DNS Query ID と Response ID の確認

プログラムが表示した ID は 10 進数で 52921，16 進数で `0xceb9` であった．Wireshark でも Query と Response の Transaction ID はいずれも `0xceb9` であり，問い合わせと応答の ID が一致した．

## 発展課題 4-3

### 課題内容

`socket_dns_test.py` が生成した DNS 問い合わせパケットの内容を確認し，DNS の規定に沿っているかを調べる．

### 実験方法

1. 4-2 で取得した pcap を Wireshark で開き，DNS Query パケットを選択する．
2. DNS ヘッダと Question セクションを展開し，ID，フラグ，質問数，問い合わせ名，問い合わせ型，クラスを確認する．
3. 指導書で指定された DNS 形式・RFC の記述と照合する．

### 結果

DNS 問い合わせパケットの画面を図 18 に示す．

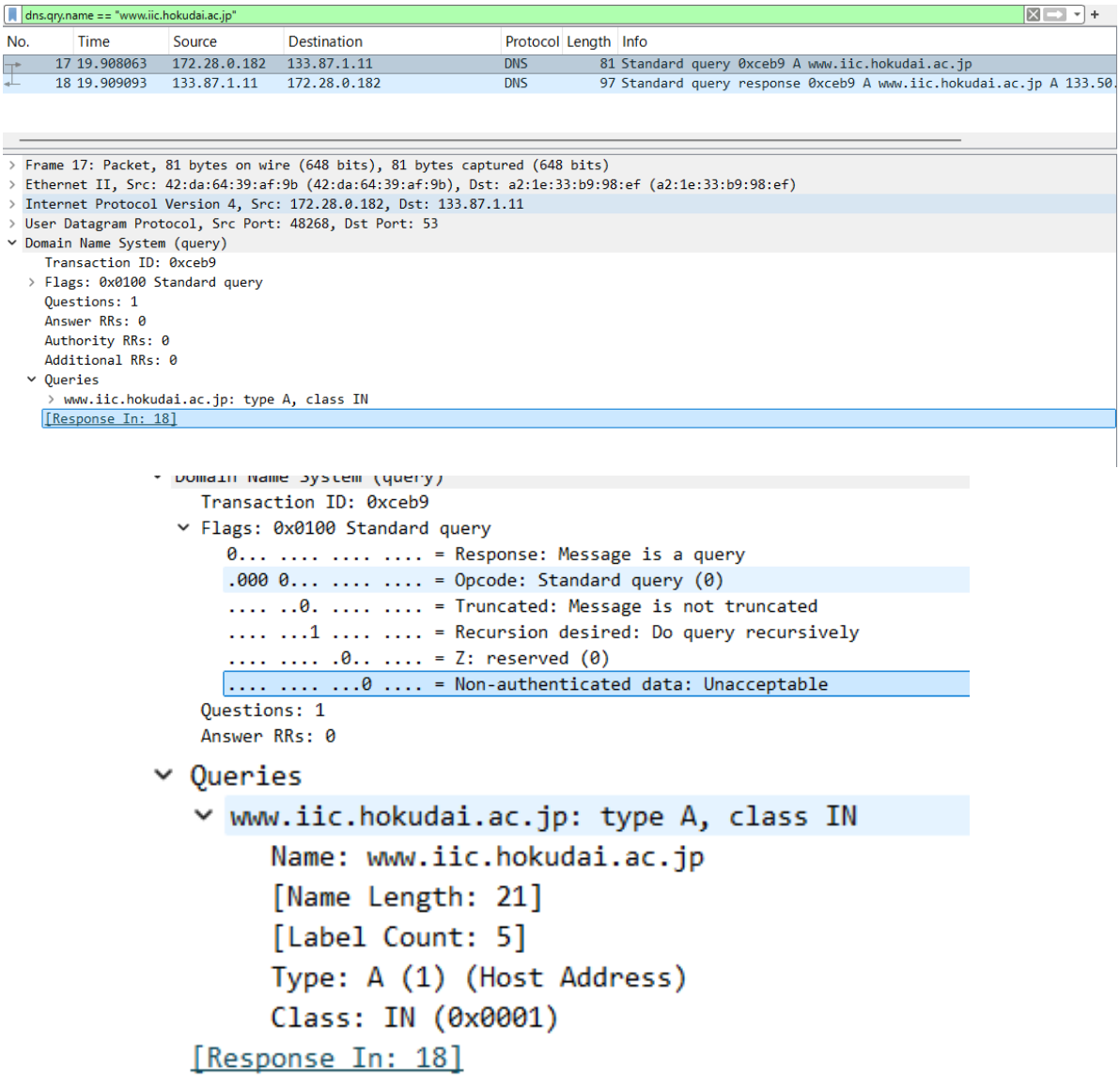






図 18: DNS 問い合わせパケットの構造

Wireshark で確認した Transaction ID は `0xceb9`，Flags は `0x0100`，Questions は 1 であり，Answer/Authority/Additional RRs はいずれも 0 であった．Flags の内訳は，QR=0（問い合わせ），Opcode=0（標準問い合わせ），TC=0（切り詰めなし），RD=1（再帰問い合わせ希望），予約ビット=0，AD=0 であった．AD ビットを問い合わせでクリアする形式は RFC 4035 §4.6 にも沿っている．Question は QNAME=`www.iic.hokudai.ac.jp`，QTYPE=A（1），QCLASS=IN（`0x0001`）である．DNS ヘッダおよび質問部の形式を定める RFC 1035 §4.1.1 – 4.1.2 と照合し，確認した値は標準問い合わせの形式に沿っていることを確認した．

## 課題 4-4

### 課題内容

TCP サーバ tcp_server.py と TCP クライアント tcp_client.py を実行し，クライアントから送信した文字列をサーバが受信することを確認する．

### 実験方法

1. サンプルをコピーした /socket で，サーバの接続先を sshd の IPv4 または IPv6 アドレスに設定して起動する．

```
cd ~/socket
./tcp_server.py
```

2. 別の演習環境で tcp_client.py を実行し，メッセージを送る．

```
cd ~/socket
./tcp_client.py
```

3. サーバ側で受信した文字列と接続元を確認し，サーバを Ctrl-C で停止する．

### 結果

サーバ側・クライアント側の実行結果を出力 29，出力 30 に示す．使用したサーバのソースコードをリスト 9 に示す．

```
#!/usr/bin/python3

from datetime import datetime
import socket


# 特定のクライアントからの通信のみを許可したい場合は '' の箇所にそのホスト名またはアドレスを記載
HOSTNAME = ''
PORT_NUM = 6789

MAX_SIZE = 1024

print('The server started at', datetime.now())
print('Waiting...')


# 1. socket.socket(), socket の作成 (ストリーム型=TCP)
# IPv4 で待ち受けする場合は AF_INET で、IPv6 で待ち受けする場合は AF_INET6 を利用すること。
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
#with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
    # 2. bind(), アドレス、ポート番号を socket に結び付ける
    server.bind((HOSTNAME, PORT_NUM))
    # 3. listen(), クライアントの待ち受け
    server.listen()
    while True:
        # 4.  accept(), クライアントの接続要求が来た時に、当該クライアント用の新たなソケット (conn) を作成
        conn, addr = server.accept()
        with conn:
            while True:
                # 5. recv(), ソケットからデータを受信。
                data = conn.recv(MAX_SIZE)
                if not data:
                    break
                print('Received text:', data.decode('utf-8'), 'from ', addr)
                # 6. sendall(), ソケットにデータを送信する
                conn.sendall(data)
        # 7. close() クライアント用のソケットを閉鎖
        conn.close()
```

ソースコード 9: tcp_server.py のソースコード

```python
#!/usr/bin/python3
from datetime import datetime
import socket

HOSTNAME = '127.0.0.1'
PORT_NUM = 6789
MAX_SIZE = 1024

print('The client started at', datetime.now())
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.connect((HOSTNAME, PORT_NUM))
    print("Input text: ", end='')
    msg = input()
    server.sendall(msg.encode('utf-8'))
    data = server.recv(MAX_SIZE)
    print('Echo back text:', data.decode('utf-8'))
```

ソースコード 10: 4-4 で使用した IPv4 版 tcp_client.py

出力 29: tcp_server.py の実行結果

```
guest@host82:~/socket$ ./tcp_server.py
The server started at 2026-11-12 00:40:38.153942
Waiting...
Received text: Test message. from  ('127.0.0.1', 55982)
```

出力 30: tcp_client.py の実行結果

```
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 00:42:33.305380
Input text: Test message.
Echo back text: Test message.
```

クライアントが送信した `Test message.` をサーバが受信し，同じ文字列がクライアントにも返された．

## 課題 4-5

### 課題内容

`tcp_client.py` の接続先を 172.28.0.200，サービスを echo (TCP ポート 7) に変更し，送信した文字列が返ることを確認する．

### 実験方法

1. `tcp_client.py` の接続先を 172.28.0.200，ポートを echo (7/tcp) に変更する．
2. プログラムを実行し，入力文字列とエコーされた文字列を記録する．

### 結果

実行に用いた tcp_client.py のソースコードをリスト 11 に，実行結果を出力 31 に示す．

```python
#!/usr/bin/python3

from datetime import datetime
import socket


#HOSTNAME = '::1'
HOSTNAME = '172.28.0.200'
PORT_NUM = 7

MAX_SIZE= 1024

print('The client started at', datetime.now())


# 1. socket の作成
# IPv4 の場合は AF_INET の行を有効化し、v6 の場合は AF_INET6 の行を有効化する
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
#with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
# 2. ソケットのサーバへの接続 (connect)
server.connect((HOSTNAME, PORT_NUM))
print("Input text: ", end='')
msg = input()
# 3. メッセージの送受信 (sendall, recv)
server.sendall(msg.encode('utf-8'))
data = server.recv(MAX_SIZE)
print('Echo back text:', data.decode('utf-8'))
server.close()
```

ソースコード 11: echo サーバ向け tcp_client.py のソースコード

出力 31: echo サーバとの通信結果

```
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 00:59:51.087378
Input text: Hello World.
Echo back text: Hello World.
```

接続先を 172.28.0.200:7 に設定し，Hello World. を送信したところ，同じ文字列が返された．

# 課題 4-6

## 課題内容

tcp_client.py の接続先を 172.28.0.200 の TCP ポート 6789 に変更し，サーバが混雑しているときにすぐ返事が返らないことを確認する．

## 実験方法

1. 接続先を 172.28.0.200，ポートを 6789 に変更した tcp_client.py を実行する．
2. 応答が返るまでの様子を記録し，演習環境の指示に従って終了する．

## 結果

使用したソースコードをリスト 12 に，実行結果を出力 32 に示す．

```python
#!/usr/bin/python3

from datetime import datetime
import socket


#HOSTNAME = '::1'
HOSTNAME = '172.28.0.200'
PORT_NUM = 6789

MAX_SIZE= 1024

print('The client started at', datetime.now())


# 1. socket の作成
# IPv4 の場合は AF_INET の行を有効化し、v6 の場合は AF_INET6 の行を有効化する
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
#with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
# 2. ソケットのサーバへの接続 (connect)
server.connect((HOSTNAME, PORT_NUM))
print("Input text: ", end='')
msg = input()
# 3. メッセージの送受信 (sendall, recv)
server.sendall(msg.encode('utf-8'))
data = server.recv(MAX_SIZE)
print('Echo back text:', data.decode('utf-8'))
server.close()
```

ソースコード 12: 混雑時の確認に用いた tcp_client.py

出力 32: TCP ポート 6789 への接続結果

```
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 01:04:07.031393
Input text: Hello message.
Echo back text: Hello message.
```

接続先 172.28.0.200:6789 に Hello message. を送信し，Hello message. が返ることを確認した．記録された出力には応答までの待ち時間が含まれていないため，応答遅延の程度はこのログからは評価できない．

## 課題 4-7

### 課題内容

IPv6 対応の 3 種類の TCP サーバ（select，fork，multithreading）を起動し，IPv6 対応クライアントから接続できることを確認する．

### 実験方法

1. 各サーバプログラムを `/socket` で順番に起動する．
2. クライアントを IPv6 対応にし，サーバの IPv6 アドレスに接続してメッセージを送信する．
3. サーバごとにクライアントとサーバ両方の出力を記録し，次のサーバを試す前に停止する．

### 結果

各サーバに対するクライアント・サーバの実行結果を出力 33〜出力 35 に示す．使用したソースコードをリスト 13〜リスト 16 に示す．

```python
#!/usr/bin/python3

from datetime import datetime
import socket
import select
import sys

HOSTNAME = ''
PORT_NUM = 6789
MAX_SIZE = 1024

print('The server started at', datetime.now())
print('Waiting...')

def accept_loop(server_sock):
  fds = [server_sock]
  while True:
    r, _, _ = select.select(fds, [], [])
    for sock in r:
      if sock == server_sock:
        new_sock, addr = sock.accept()
        fds.append(new_sock)
      else:
        data = sock.recv(MAX_SIZE)
        if not data:
          break
        print('Received text:', data.decode('utf-8'), 'from', addr)
        sock.sendall(data)
        sock.close()
        fds.remove(sock)

if __name__ == '__main__':
  server = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
  #server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  server.bind((HOSTNAME, PORT_NUM))
  server.listen()
  try:
    accept_loop(server)
  except KeyboardInterrupt:
    server.close()
```

ソースコード 13: IPv6 対応 tcp_server_select.py

```python
#!/usr/bin/python3

from datetime import datetime
import socket
import os

HOSTNAME = ''
PORT_NUM = 6789
MAX_SIZE = 1024

print('The server started at', datetime.now())
print('Waiting...')

#with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
  server.bind((HOSTNAME, PORT_NUM))
  server.listen()
  try:
    while True:
      client, addr = server.accept()
      child_pid = os.fork()
      if child_pid == 0:
        data = client.recv(MAX_SIZE)
        if data:
          print('Received text:', data.decode('utf-8'), 'from', addr)
          client.sendall(data)
          client.close()
  except KeyboardInterrupt:
    pass
```

ソースコード 14: IPv6 対応 tcp_server_fork.py

```python
#!/usr/bin/python3

from datetime import datetime
import socket
import threading

alive = True
observe_thread = None
HOSTNAME = ''
PORT_NUM = 6789
MAX_SIZE = 1024

print('The server started at', datetime.now())
print('Waiting...')

def client_handler(client, addr):
  data = client.recv(MAX_SIZE)
  if data:
    print('Received text:', data.decode('utf-8'), 'from', addr)
    client.sendall(data)
    client.close()

if __name__ == '__main__':
  #with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
  with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
    server.bind((HOSTNAME, PORT_NUM))
    server.listen()
    try:
      while True:
        client, addr = server.accept()
        thread = threading.Thread(target = client_handler, args = (client, addr))
        thread.start()
        while thread.is_alive():
          thread.join()
    except KeyboardInterrupt:
      alive = False
      pass
```

ソースコード 15: IPv6 対応 tcp_server_multithreading.py

```python
#!/usr/bin/python3

from datetime import datetime
import socket

HOSTNAME = '::1'
#HOSTNAME = '172.28.0.200'
PORT_NUM = 6789
MAX_SIZE= 1024

print('The client started at', datetime.now())

#with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as server:
    server.connect((HOSTNAME, PORT_NUM))
    print("Input text: ", end='')
    msg = input()
    server.sendall(msg.encode('utf-8'))
    data = server.recv(MAX_SIZE)
    print('Echo back text:', data.decode('utf-8'))
    server.close()
```

ソースコード 16: IPv6 対応 tcp_client.py

出力 33: select サーバの実行結果

```
guest@host82:~/socket$ ./tcp_server_select.py
The server started at 2026-11-12 01:11:10.323441
Waiting...
Received text: How are you doing. from ('::1', 41992, 0, 0)
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 01:11:31.124926
Input text: How are you doing.
Echo back text: How are you doing.
```

出力 34: fork サーバの実行結果

```
guest@host82:~/socket$ ./tcp_server_fork.py
The server started at 2026-11-12 01:16:08.792394
Waiting...
Received text: Text. from ('::1', 39216, 0, 0)
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 01:16:30.536484
Input text: Text.
Echo back text: Text.
```

出力 35: multithreading サーバの実行結果

```
guest@host82:~/socket$ ./tcp_server_multithreading.py
The server started at 2026-11-12 01:25:56.407150
Waiting...
Received text: Test message. from ('::1', 43042, 0, 0)
guest@host82:~/socket$ ./tcp_client.py
The client started at 2026-11-12 01:26:39.304585
Input text: Test message.
Echo back text: Test message.
```

IPv6 ループバック::1 を用いて 3 種類のサーバに接続した．select サーバでは How are you doing.，fork サーバでは Text.，multithreading サーバでは Test message. を送信し，それぞれ同じ文字列がクライアントに返り，サーバ側にも接続元::1 と受信文字列が表示された．

## 課題 4-8

### 課題内容

tcp_server_fork.py の実行中に ps コマンドでプロセス一覧を確認し，クライアント接続時にプロセス数が増えることを確かめる．

### 実験方法

1. IPv6 対応に変更した tcp_server_fork.py を起動し，クライアントから接続する．
2. クライアントとの通信後，別端末から ps を実行してサーバの親子プロセスを確認する．

### 結果

クライアントとの通信後に実行した ps の出力を出力 36 に示す．

出力 36: クライアント接続後のプロセス一覧

```
guest@host82:~/socket$ ps -ef | grep tcp_server_fork.py
guest       1746    1676  0 01:16 pts/0    00:00:00 /usr/bin/python3 ./tcp_server_fork.py
guest       1748    1746  0 01:16 pts/0    00:00:00 /usr/bin/python3 ./tcp_server_fork.py
guest       1750    1703  0 01:24 pts/1    00:00:00 grep --color=auto tcp_server_fork.py
```

クライアント接続後のプロセス一覧では，PID 1746 のサーバプロセスに加えて，その子プロセス PID 1748 が確認できた．子プロセスの PPID は 1746 であり，クライアント接続を処理するために fork で子プロセスが生成されたことが分かる．接続前の一覧は記録していないため，ここでは接続後の親子関係を示した．